



PYTHON

Types de variables, conditions et fonctions

Cours, méthodes et exercices corrigés — NSI / Première

Ce que tu vas savoir faire à la fin de ce chapitre

Reconnaître et manipuler les types de base : nombres entiers, décimaux, textes, booléens.

Écrire des conditions (if / elif / else) pour faire prendre des décisions à un programme.

Créer et utiliser tes propres fonctions pour organiser et réutiliser ton code.

Comment utiliser ce document ?

 = astuce •  = piège classique •  = exemple pas à pas •  = à toi de jouer

Sommaire

- Chapitre 1 — Les types de variables et les chaînes de caractères
- Chapitre 2 — Les conditions
- Chapitre 3 — Les fonctions
- Annexe — Corrigés et auto-évaluation

Chapitre 1 — Les types de variables

1. Une variable, c'est quoi ?

Imagine une boîte sur laquelle tu colles une étiquette : c'est ta variable. À l'intérieur, tu ranges quelque chose — un nombre, un mot, une réponse oui/non. Le type, c'est simplement la nature de ce que contient la boîte : Python a besoin de le savoir pour comprendre ce qu'il a le droit de faire avec (additionner, découper, comparer...).



À retenir

Chaque valeur en Python possède un type, et la fonction `type(valeur)` permet de le connaître directement.

2. Les quatre types de base

Type	Nom Python	Exemple	Ce que c'est
Nombre entier	int	12 -5 1000	Un nombre sans virgule
Nombre décimal	float	3.14 -0.5 1e10	Un nombre à virgule (avec un point !)
Chaîne de caractères	str	"Bonjour" 'NSI'	Du texte, entre guillemets
Booléen	bool	True False	Une valeur logique : vrai ou faux



Piège classique n°1 : la virgule française

En français on écrit 3,14 mais Python attend un point : 3.14. Écrire 3,14 crée en réalité deux valeurs séparées par une virgule (un tuple), ce qui provoque une erreur ou un résultat inattendu.



Exemple pas à pas — découvrir `type()`

Voici un petit programme qui affiche le type de plusieurs valeurs (extrait de l'exercice 1) :

```
for element in ("Exercice", 123., True, 1e10, "False", '3,14'):  
    print(element, "->", type(element))
```

Ce que Python affiche pour chaque élément :

Valeur écrite	Type obtenu	Pourquoi
"Exercice"	str	Du texte entre guillemets
123.	float	Le point après 123 en fait un décimal (123.0)
True	bool	Un mot-clé booléen de Python

Valeur écrite	Type obtenu	Pourquoi
1e10	float	Écriture scientifique : toujours un décimal
"False"	str	Attention : entre guillemets, ce n'est plus un booléen mais du texte !
'3,14'	str	La virgule empêche Python d'y voir un nombre : c'est juste du texte
 Astuce "False" (avec des guillemets) et False (sans guillemets) n'ont rien à voir : le premier est une chaîne de caractères, le second est un booléen. Les guillemets changent tout !		

Exercice corrigé — traquer les erreurs de type

Un élève avait écrit ce programme pour calculer l'aire d'un cercle, mais il contenait plusieurs erreurs. Les repérer est un excellent exercice pour bien comprendre les types :

Les erreurs à corriger et pourquoi

3,14 → 3.14 : en Python, un nombre décimal s'écrit avec un point, jamais une virgule.

aire-cercle → aire_cercle : le tiret est interprété comme une soustraction ; les noms de variables et de fonctions utilisent le tiret bas (underscore) _.

Parenthèse manquante après def aire_cercle(rayon : il faut fermer la parenthèse avant les deux-points.

r → rayon : mieux vaut utiliser un nom de paramètre explicite, plus facile à relire.

La formule doit être pi * rayon ** 2 (rappel : ** est la puissance en Python, pas ^).

print(aire_cercle(10)) : il manquait la parenthèse fermante du print.

Version corrigée et fonctionnelle :

```
pi = 3.14

def aire_cercle(rayon):
    return pi * rayon ** 2

print(aire_cercle(10))  # affiche 314.0
```

3. Les chaînes de caractères (str) en détail

Une chaîne de caractères est une suite de caractères (lettres, chiffres, espaces, symboles). Chaque caractère a une position, appelée un indice, qui commence toujours à 0.

Caractère	U	N	E	X	E	M	P	L	E
Indice	0	1	2	3	4	5	6	7	8

Les instructions de base sur les chaînes

Commande	Effet
ch = input(question)	Ouvre une fenêtre de saisie et stocke le texte tapé dans ch
len(ch)	Renvoie la longueur (le nombre de caractères) de ch
ch[i]	Renvoie le caractère à la position i (le premier est à l'indice 0)
mots[debut:fin]	Découpe : renvoie les caractères de debut (inclus) à fin (exclu)
mots1 + mots2	Concatène (colle bout à bout) deux chaînes

Commande	Effet
mots * nb	Recopie la chaîne nb fois
mots.upper() / .lower()	Renvoie la chaîne en majuscules / en minuscules
str(nb)	Transforme un nombre en chaîne de caractères
eval(ch)	Transforme une chaîne en valeur numérique

⚠ Piège classique n°2 : une chaîne ne se modifie pas caractère par caractère

ch[2] = 'A' provoque une erreur ! Une chaîne de caractères ne peut pas être modifiée directement : il faut en reconstruire une nouvelle (par exemple avec du découpage et de la concaténation).

Rechercher et remplacer dans une chaîne

Commande	Effet
l in ch	Renvoie True si la lettre l apparaît dans la chaîne ch
ch.count(texte)	Compte le nombre de fois où texte apparaît dans ch
ch.replace(txt1, txt2)	Remplace chaque txt1 trouvé dans ch par txt2
ch.find(texte)	Renvoie la position de texte dans ch, ou -1 s'il n'apparaît pas

✏ À toi de jouer — s'entraîner sur les chaînes

Ces cinq mini-programmes se construisent tous avec les outils du tableau ci-dessus. Ils vont du plus simple au plus complet — un bon échauffement avant les fonctions du chapitre 3 :

- Programme 1 : écrire compte(texte, lettre) qui compte le nombre d'apparitions de lettre dans texte.
- Programme 2 : écrire retourne(mot) qui renvoie le mot écrit à l'envers (ex : retourne("NSI") → "ISN").
- Programme 3 : écrire palyndrome(nom) qui renvoie True si le mot est un palindrome, False sinon.
- Programme 4 : écrire presence(mot, lettre) qui renvoie True si lettre apparaît dans mot (jeu du pendu).
- Programme 5 : écrire melangemot(mot) qui renvoie le mot avec ses lettres mélangées.

🔍 Corrigé guidé — Programme 1 : compte(texte, lettre)

L'idée : on parcourt chaque caractère du texte avec une boucle, et on incrémente un compteur à chaque fois qu'il correspond à la lettre recherchée.

```
def compte(texte, lettre):
    nb = 0
    for car in texte:
        if car == lettre:
            nb = nb + 1
    return nb

print(compte("anticonstitutionnellement", "n")) # affiche 5
```

Les programmes 2 à 5 se construisent sur le même principe : une boucle qui parcourt la chaîne (ou la méthode .count() pour aller plus vite), combinée à ce que tu sais déjà sur le découpage et les tests. Essaie-les avant de regarder les corrigés en annexe !

Chapitre 2 — Les conditions

1. Une condition, qu'est-ce que c'est ?

Un programme doit souvent s'adapter à une situation : afficher un message différent selon l'âge d'une personne, autoriser ou refuser un accès... Une condition est une expression logique dont le résultat est soit Vrai (True), soit Faux (False).



À retenir

Une condition renvoie toujours un booléen (True ou False), exactement comme une réponse à une question fermée.

2. Les opérateurs de comparaison

Opérateur	Signification	Exemple
==	égal à	a == 5
!= ou <>	différent de	a != 5
< et >	inférieur à / supérieur à	a < 10
<= et >=	inférieur ou égal / supérieur ou égal	a >= 0



Piège classique : = n'est pas ==

a = 5 donne la valeur 5 à la variable a (une affectation). a == 5 compare a à 5 et renvoie True ou False (un test). Confondre les deux est l'une des erreurs les plus fréquentes en début d'apprentissage !

3. Combiner des conditions : and, or, not

Lorsque la situation est plus compliquée, on combine plusieurs conditions grâce à des opérateurs logiques :

- and (« et ») : vrai seulement si les deux conditions sont vraies en même temps.
- or (« ou ») : vrai dès qu'au moins une des deux conditions est vraie.
- not (« non ») : inverse le résultat d'une condition (True devient False, et inversement).



Exemple pas à pas — tester and / or à la main

Voici un extrait de l'exercice 4, qui teste plusieurs combinaisons logiques pour x = 0, x = 10 puis x = -20 :

```
for x in (0, 10, -20):
    print(x < 10 and x > -10)
    print(x < -10 or x > 10)
```

Détail du raisonnement pour chaque valeur de x :

x	x < 10 and x > -10	x < -10 or x > 10
0	True and True → True	False or False → False
10	False and True → False	False or False → False
-20	True and False → False	True or False → True



Méthode pour lire une condition composée

Évalue chaque petite condition séparément (elle donne True ou False), puis applique l'opérateur logique sur les résultats obtenus — exactement comme dans le tableau ci-dessus.

4. Les structures conditionnelles

a. if ... : exécuter un bloc si une condition est vraie

```
if membre == "oui":
    print("Accès autorisé")
```

b. if ... else ... : un bloc si vrai, un autre si faux

```
if membre == "oui":
    print("Accès autorisé")
else:
    print("Accès refusé")
```

c. if ... elif ... else ... : gérer plusieurs cas

elif est la contraction de else if, c'est-à-dire « sinon si ». Les conditions sont testées dans l'ordre, et dès qu'une est vraie, son bloc s'exécute (les suivantes ne sont plus regardées).

```
if statut == "secret":
    print("seuls les membres voient le groupe et les publications")
elif statut == "Fermé":
    print("Tout le monde voit le groupe mais pas les publications")
else:
    print("Tout le monde voit le groupe et les publications")
```



Deux règles de syntaxe à ne jamais oublier

Une condition (if / elif / while) doit toujours être suivie de deux-points :

Les instructions à exécuter si la condition est vérifiée doivent être indentées (décalées vers la droite) — c'est cette indentation, et non des accolades, qui délimite le bloc en Python.



Exemple pas à pas — quatre variantes du même test (exercice 5)

Ces quatre petits programmes montrent bien la différence entre tester une condition directement, la comparer à False, ou utiliser un if...else :

Program me	Code	Résultat
1	if (12*2 == 24): print("Logique.")	12*2 == 24 est True → affiche "Logique."
2	if (12*2 == 24) == False: print("Logique.")	True == False est False → rien ne s'affiche
3	if (12*2 == 23) == False: ... puis print("Ou pas.")	12*2==23 est False, donc False==False est True → affiche "Logique." puis, hors du if, "Ou pas."
4	if (12*2 == 23): ... else: print("Ou pas.")	La condition est fausse → le bloc else s'exécute → affiche "Ou pas."

5. Entraîne-toi

Les corrigés de ces exercices se trouvent en annexe, à la fin du document — essaie d'abord par toi-même !

- 18 — Donner deux valeurs de y pour lesquelles $(y < 3 \text{ and } y == 2)$ or $(y > 5 \text{ and } y != 6)$ est vraie.
- 19 — Écrire en Python : « A est égal à 5 ou à 6 et B est strictement inférieur à 4 ».
- 20 — Que vaut l'expression logique $6 < 8 \text{ and } 2 < 1$?
- 21 — Qu'affiche ce programme si $x = 112$, puis si $x = 0$? `(if x==112: print("C'est le bon code") ; print("Au revoir"))`
- 23 — Écrire la suite d'instructions qui affiche une alerte si la variable `temperature` dépasse 40 °C.
- 24 — Qu'affiche ce programme si $x = 100$, puis si $x = 200$? `(if x<=150: print(x+10) ; else: print(x-30))`
- 25 — Deux if imbriqués avec un else : qu'affiche le programme pour $x = 150$ puis $x = 300$? Que se passe-t-il si l'on remplace le second if par un elif ?
- 26 — Trouver le bug : un programme mélange `=` et `==`, et un mauvais niveau d'indentation. Corrige-le.

Chapitre 3 — Les fonctions

1. Pourquoi utiliser des fonctions ?

Imagine une petite machine : tu lui donnes un ou plusieurs ingrédients (les paramètres), elle effectue un traitement, et elle te renvoie un résultat. C'est exactement ce qu'est une fonction : un morceau de programme, écrit une seule fois, que l'on peut réutiliser autant de fois que l'on veut, avec des ingrédients différents à chaque fois.



Fonction ou procédure ?

Dans certains langages, on distingue les fonctions (qui renvoient une valeur en sortie) des procédures (qui effectuent un traitement sans rien renvoyer, par exemple un simple affichage). En Python, tout s'écrit avec `def`, que la fonction renvoie quelque chose ou non.

2. Définir une fonction

On définit une fonction avec le mot-clé `def`, suivi du nom de la fonction, de ses paramètres entre parenthèses, puis de deux-points. Le corps de la fonction est indenté, comme pour un `if`.

```
def bonjour():  
    print("Bonjour !")
```

Une fonction peut prendre des valeurs en entrée : ce sont les paramètres. Ils ne sont utilisés qu'à l'intérieur de la fonction — on parle de variables locales.

```
def produit(a, b):  
    return a * b
```

3. Renvoyer un résultat avec `return`

L'instruction `return` indique ce que la fonction doit renvoyer. Si `return` n'est pas utilisé, ou utilisé sans argument, la fonction renvoie un objet vide : `None`.



Piège classique : oublier le `return`

Une fonction qui ne fait qu'un `print()` à l'intérieur, sans `return`, ne renvoie rien d'utilisable. Si tu écris `resultat = maFonction(...)` et que `maFonction` n'a pas de `return`, `resultat` vaudra `None` — une source d'erreurs très fréquente !

4. Appeler une fonction

Dans le programme principal, on appelle une fonction en écrivant son nom suivi des valeurs des paramètres entre parenthèses. Une même fonction peut être appelée plusieurs fois, avec des valeurs différentes à chaque fois :

```
def tempsTrajet(distance, vitesse):  
    return distance / vitesse  
  
temps1 = tempsTrajet(200, 110)  
print(temps1)  
  
temps2 = tempsTrajet(150, 80)  
print(temps2)
```

5. Variables locales et variables globales

C'est un point essentiel — et souvent source de confusion. Une variable définie à l'intérieur d'une fonction (variable locale) n'existe que pendant l'exécution de cette fonction. Une variable définie dans le programme principal (variable globale) peut être lue depuis une fonction, mais pas modifiée directement.

Cas n°1 : Python va chercher la variable globale

```
def coucou(a):  
    return a + n  
  
n = 5  
print(coucou(3))    # affiche 8
```

Python ne connaît pas `n` à l'intérieur de la fonction, alors il va la chercher dans les variables globales : il trouve `n = 5`, donc `a + n = 3 + 5 = 8`.

Cas n°2 : la variable locale prend le dessus

```
def coucou(a):  
    n = 1  
    return a + n  
  
n = 5  
print(coucou(3))    # affiche 4  
print(n)            # affiche 5
```

Cette fois, Python trouve une variable `n` définie à l'intérieur de la fonction (`n = 1`) : c'est celle-là qu'il utilise pour le calcul (`3 + 1 = 4`). Cette variable est locale : dès qu'on sort de la fonction, `n` reprend sa valeur globale (5).

Deux façons de faire ressortir une valeur modifiée

- 1) `return` : la fonction renvoie la ou les nouvelles valeurs, que l'on récupère à l'appel (`a, m = coucou(3)`).
- 2) `global` : le mot-clé `global` `n, m` à l'intérieur de la fonction indique que l'on modifie directement les variables globales `n` et `m`, sans avoir besoin de `return`.

```
# Méthode return  
def coucou(a):  
    return a + n, m - a  
  
n = 5  
m = 12  
n, m = coucou(3)  
print(n, m)          # affiche 8 9  
  
# Méthode global  
def coucou(a):  
    global n, m  
    n = a + n  
    m = m - a  
  
n = 5  
m = 12  
coucou(3)  
print(n, m)          # affiche 8 9
```

6. Exemple concret — la factorielle (exercice 8)

On veut calculer $n!$ (factorielle de n), c'est-à-dire le produit de tous les entiers de 1 à n . Ici encore, une fonction bien construite rend le calcul réutilisable pour n'importe quelle valeur de n :

```
def factorielle(n):
    resultat = 1
    for i in range(2, n + 1):
        resultat *= i
    return resultat

n = 5
print(f"{n}! =", factorielle(n))    # affiche 5! = 120
```

Déroulé pas à pas pour $n = 5$ (la variable `resultat` est locale à la fonction) :

Tour de boucle	i	resultat avant	resultat après (resultat *= i)
1	2	1	$1 \times 2 = 2$
2	3	2	$2 \times 3 = 6$
3	4	6	$6 \times 4 = 24$
4	5	24	$24 \times 5 = 120$

7. Exemple concret — la division euclidienne « à la main » (exercice 9)

On veut retrouver le quotient n et le reste a de la division de `a_initial` par `b`, mais sans utiliser `//` ni `%` : uniquement en soustrayant `b` autant de fois que possible.

```
n = 0
a = 27
b = 5
a_initial = a
while a >= b:
    n = n + 1
    a = a - b

print("a final =", a)      # 2
print("n =", n)            # 5
```

On vérifie ensuite que $27 = 5 \times 5 + 2$, et que le reste (2) est bien compris entre 0 et `b` exclu — c'est la définition même d'une division euclidienne :

```
if a_initial == b * n + a and 0 <= a < b:
    print("Le résultat est bien une division euclidienne.")
```



Pourquoi une boucle `while` et pas une boucle `for` ici ?

On ne connaît pas à l'avance le nombre de soustractions nécessaires (cela dépend de `a_initial` et de `b`) : c'est donc une boucle non bornée (`while`), qui s'arrête dès que la condition `a >= b` devient fausse.

8. Auto-évaluation — variables locales, globales et return

Le petit questionnaire ci-dessous (12 questions) revient sur les pièges les plus fréquents autour des fonctions : que se passe-t-il si on modifie un paramètre à l'intérieur d'une fonction ? Que renvoie une fonction sans `return` ? Les réponses et explications sont en annexe.

- Q1-Q2-Q3-Q4 — Une fonction `ChangeChaine(x)` modifie `x` et parfois fait un `return (x)`. Que s'affiche-t-il selon les cas, pour `print(y)`, `print(ChangeChaine(y))` ou `y = ChangeChaine(y)` ?
- Q5 à Q9 — Une fonction `operation(u, v, w)` calcule $u + 2v + 3w$. Que valent les variables après l'appel, et le nom d'une variable interne à la fonction a-t-il un impact à l'extérieur ?
- Q10-Q11 — Deux fonctions (`add` et `operation`) s'appellent l'une l'autre : que renvoie le programme ?
- Q12 — Une fonction `div_euclide(x, y)` renvoie $(x \% y, x // y)$: que renvoie `div_euclide(8, 5)` ?

Annexe — Corrigés

Corrigés — chaînes de caractères (chapitre 1)

Programme 2 — `retourne(mot)` : on utilise le découpage avec un pas négatif pour lire la chaîne à l'envers.

```
def retourne(mot):  
    return mot[::-1]
```

Programme 3 — `palyndrome(nom)` : un mot est un palindrome s'il est identique à son écriture inversée.

```
def palyndrome(nom):  
    return nom == retourne(nom)
```

Programme 4 — `presence(mot, lettre)` : l'opérateur `in` fait tout le travail.

```
def presence(mot, lettre):  
    return lettre in mot
```

Programme 5 — `melangemot(mot)` : on transforme le mot en liste de lettres, on mélange, puis on recolle.

```
from random import shuffle  
  
def melangemot(mot):  
    lettres = list(mot)  
    shuffle(lettres)  
    return "".join(lettres)
```

Corrigés — conditions (chapitre 2)

N°	Corrigé
18	<code>y = 2</code> convient : <code>(y<3 and y==2)</code> est vraie. <code>y = 7</code> convient aussi : <code>(y>5 and y!=6)</code> est vraie. Toute valeur de <code>y</code> strictement supérieure à 5 et différente de 6 fonctionne également.
19	<code>(A == 5 or A == 6) and B < 4</code>
20	<code>6<8 and 2<1</code> → <code>True and False</code> → <code>False</code>
21	Pour <code>x = 112</code> : affiche "C'est le bon code" puis "Au revoir" (le second <code>print</code> n'est pas dans le bloc <code>if</code> , il s'exécute donc toujours). Pour <code>x = 0</code> : affiche seulement "Au revoir".
24	Pour <code>x = 100</code> : <code>100 <= 150</code> est vrai → affiche 110. Pour <code>x = 200</code> : <code>200 <= 150</code> est faux → affiche 170.

Exercice 23 :

```
if temperature > 40:  
    print("Alerte : température trop élevée !")
```

Exercice 25 — deux if imbriqués, puis `if...elif...else`

Avec le programme original (un `if` suivi d'un second `if...else` indépendant) :

- `x = 150` : le premier `if (x>100)` est vrai → "x est grand". Le second bloc est testé séparément : `x>200` est faux → le `else` s'exécute → "x est petit". Les deux messages s'affichent.
- `x = 300` : "x est grand", puis `x>200` est vrai → "x est très grand". Les deux messages s'affichent.

En remplaçant le second if par elif (on obtient une seule chaîne if / elif / else) :

- $x = 150$: $x > 100$ est vrai → seul "x est grand" s'affiche ; le elif et le else ne sont plus examinés du tout, car une branche a déjà été retenue.
- $x = 300$: même chose → seul "x est grand" s'affiche, alors qu'on aurait pu s'attendre à "x est très grand".



La leçon à retenir de cet exercice

Dans une chaîne if / elif / else, une seule branche s'exécute : dès que la première condition vraie est trouvée, Python arrête d'examiner les suivantes. Deux if séparés, eux, sont toujours testés l'un après l'autre, indépendamment.

Exercice 26 — trouver et corriger le bug

Le bug : elif note=10 utilise = (affectation) au lieu de == (comparaison), ce qui provoque une erreur de syntaxe.

Version corrigée et correctement indentée :

```
if note < 10:
    print("Recommencez le test !")
elif note == 10:
    print("Vous avez la moyenne")
else:
    print("Test réussi")
```

Corrigés — auto-évaluation sur les fonctions (chapitre 3)

N°	Réponse	Explication express
1	a — test / retest	x est un paramètre local ; return (x) renvoie bien la nouvelle valeur.
2	b — test / test	Le retour de ChangeChaine(y) n'est récupéré par personne : y n'est jamais modifié.
3	b — test / test	Sans return, la fonction ne renvoie rien d'utile, et x reste de toute façon local.
4	d — test / None	Sans return, la fonction renvoie None ; y=ChangeChaine(y) écrase donc y avec None.
5	c — 20	$2 + 2 \times 3 + 3 \times 4 = 2 + 6 + 12 = 20$.
6	c — (20, 0)	res=20 renvoyé par return ; le x global (0) n'est pas modifié par le x local de la fonction.
7	c — 0	p=20 à l'intérieur de operation() est une variable locale : elle n'affecte pas le p global.
8	b — 10	$0 + 2 \times 2 + 3 \times 2 = 0 + 4 + 6 = 10$.
9	b — (6, 1)	Avec (1,1,1) : $x=1+2+3=6$ et $y=1$. Les variables globales u, v, w ne sont pas utilisées ici.
10	b — (3, 2)	$y=add(1,1)=2$; $x=add(2,1)=3$; renvoyé sous la forme $(x,y) = (3,2)$.
11	b — (3, 2)	Avec (1,2,1) : $y=add(1,1)=2$; $x=add(2,1)=3 \rightarrow (x,y)=(3,2)$. Les variables globales ne sont pas utilisées.
12	e — (3, 1)	$8 \% 5 = 3$ (le reste) et $8 // 5 = 1$ (le quotient).



Le fil rouge à retenir de ces trois chapitres

Le type d'une valeur détermine ce que l'on a le droit d'en faire.

Une condition renvoie toujours un booléen, et and / or / not permettent d'en combiner plusieurs.

Une fonction isole un traitement dans une boîte à paramètres, et return est le seul moyen sûr d'en faire ressortir un résultat.